

7. Build a CNN model to classify cat and dog image

Aim:

Building a CNN model to classify cat and dog image

Objective:

- To understand CNN implementation using Pytorch layers
- To preprocess cat/dog images using torchvision transforms
- To train the model using Data loader, loss function and optimizer
- To evaluate model performance of CNN model using accuracy and loss metrics

Pseudo code:

Start

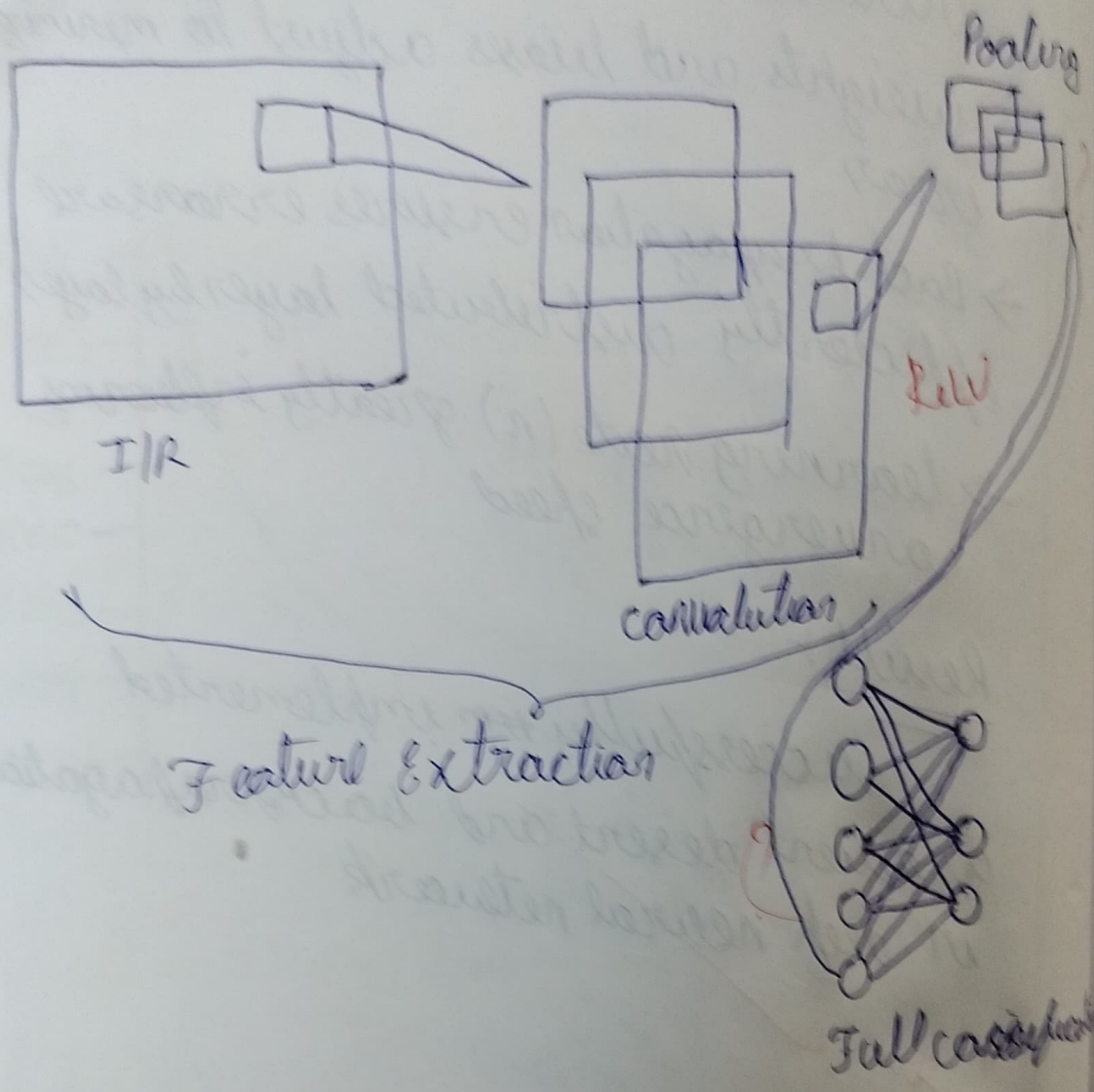
1. Import necessary libraries (tensorflow, keras, numpy, matplotlib)

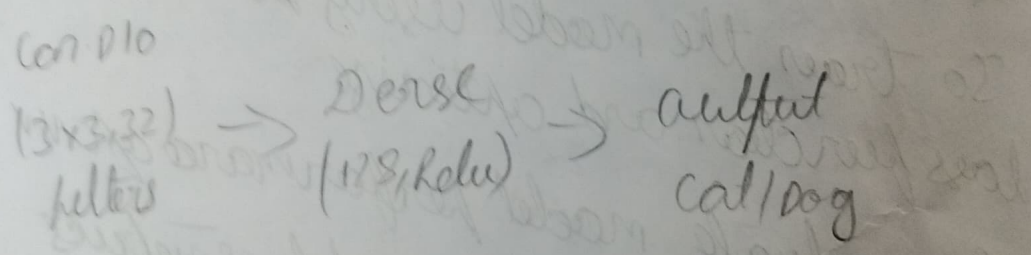
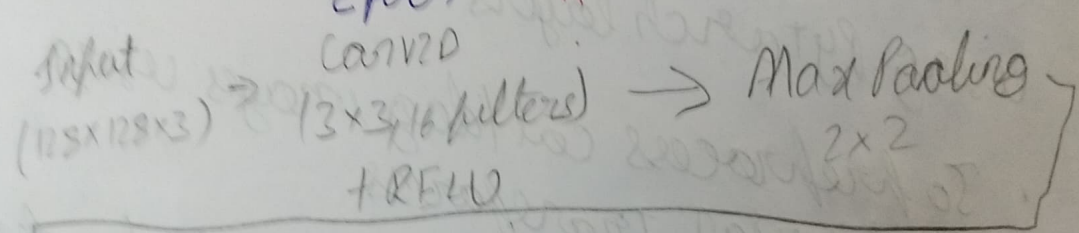
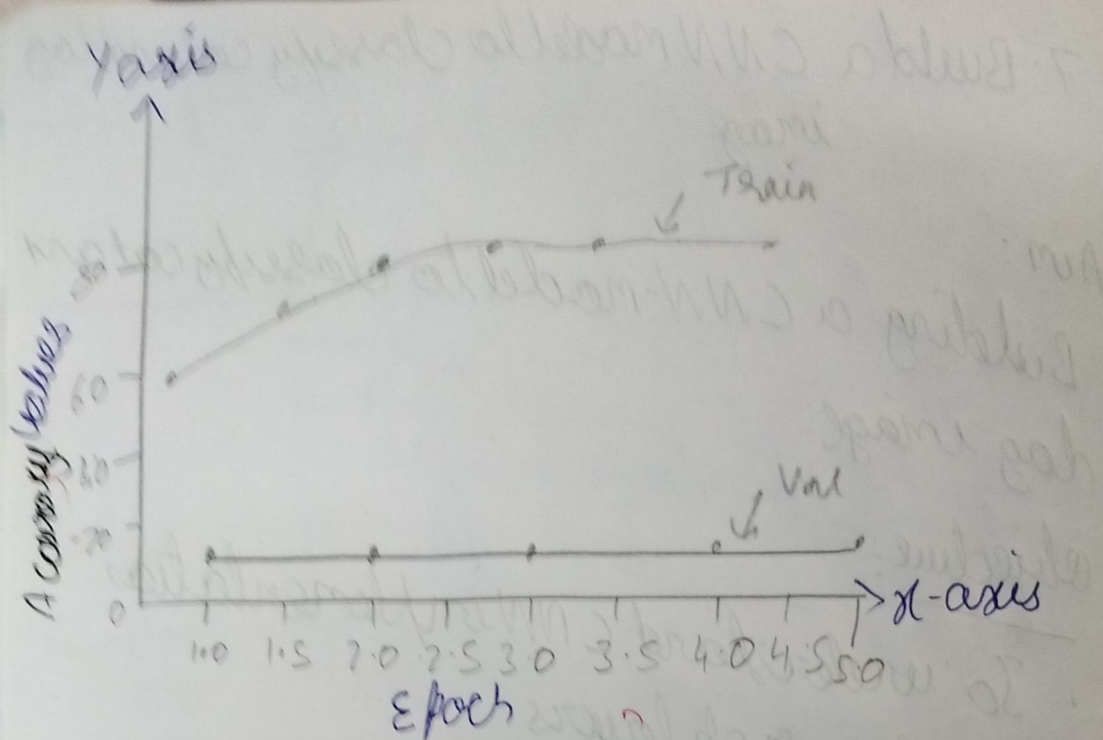
2. Load dataset

3. - Apply preprocessing: resize, normalize pixel values

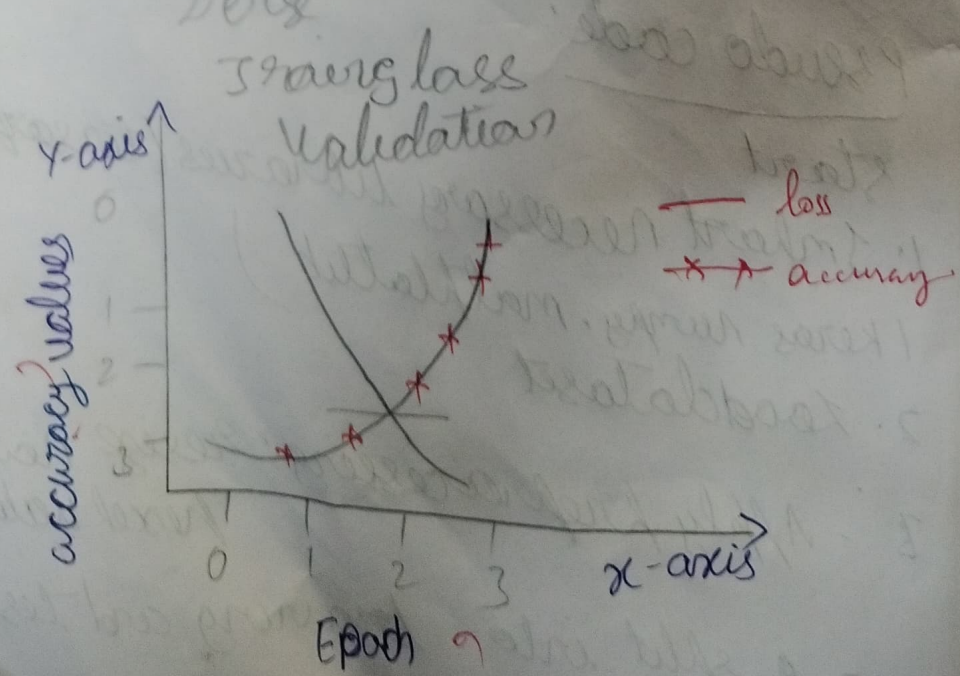
- split into training and testing

CNN architecture





Input → Conv2D → Filter
Dense



3. Define CNN architecture

- CONV2D layer + ReLU activation
- Maxpooling 2D layer
- CONV2D layer + ReLU activation
- Maxpooling 2D layer
- Flatten layer
- Dense (fully connected) hidden layer + ReLU
- Dense output layer + sigmoid/binary

4. Compile the model

- optimizer: Adam
- loss: Binary Cross Entropy
- metrics: Accuracy

5. Train the model on training data with validation split

6. Evaluate model on training data or test data

7.) Display training and validation accuracy loss curves

Epoch	Accuracy	loss	Val-accuracy	Val-loss
1/10	0.6443	0.9051	0.6909	0.6402
2/10	0.6957	0.6159	0.7636	0.4474
3/10	0.8076	0.3842	0.7729	0.4776
4/10	0.7987	0.4199	0.7445	0.5289
5/10	0.8501	0.3423	0.7455	0.4940
6/10	0.8345	0.3607	0.8364	0.3750
7/10	0.7897	0.4351	0.8091	0.4359
8/10	0.8389	0.3291	0.7727	0.4390
9/10	0.8389	0.3139	0.8182	0.4785
10/10	0.8546	0.3006	0.7909	0.4282

Observation:

- Training accuracy increases with epochs while validation accuracy
- Loss decreases as training progresses
- The CNN learns to extract features like edges, textures and shapes to differentiate cats from dogs

Result:

Successfully built and trained a CNN model that classifies images of cats and dogs

